
Fundamentals of Deep Learning, Perceptron

— ~ Dr. Anuradha Yenikar —

Unit 1: Syllabus

Fundamentals of Deep Learning: Introduction to Deep Learning, Neuron, Neural Networks, Types of Deep Learning Networks, Working of Neural Network

Perceptrons: Single Layer Perceptron, Multilayer Perceptrons (MLPs), Representation Power of MLPs, Working of Perceptron with Weights and Bias, Sigmoid Neurons, Advantage and challenges of deep learning.

Activation Functions: Activation Functions for Neural Networks, RELU, LRELU, ERELU, Sigmoid, Tanh, Softmax

Training Neural Networks: Backpropagation and Forward propagation.

Gradient Descent - Gradient Descent (GD), Vanishing Gradient Problem, Momentum Based GD, Nesterov Accelerated GD, Stochastic GD, AdaGrad, RMSProp

Hyper parameters: Layer size, Magnitude (momentum, learning rate)

What is “Deep Learning”

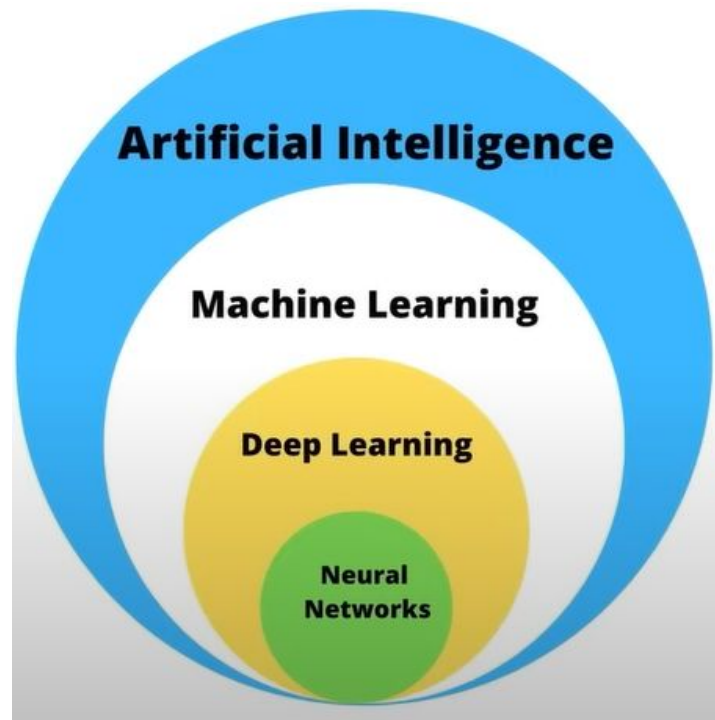


Deep Meditation?

Ofcourse NOT!

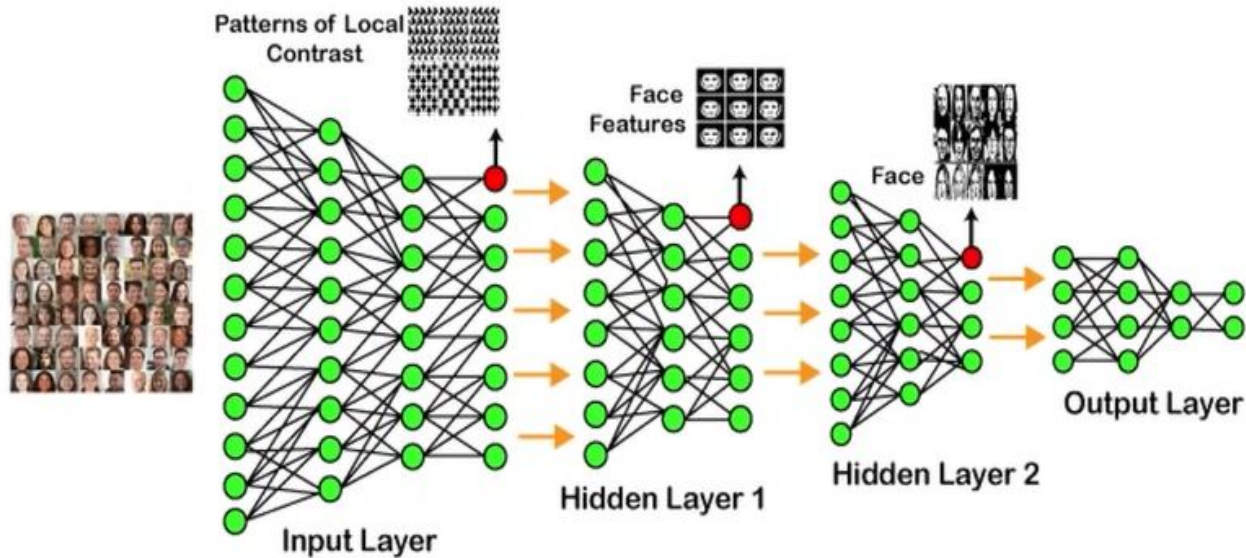
Deep Learning

- Deep Learning is a subset of ML that uses mathematical functions to map the input to the output.
- These functions can extract non-redundant information or patterns from the data, which enables them to form a relationship between the input and the output.
- This is known as learning, and the process of learning is called training.



Deep Learning

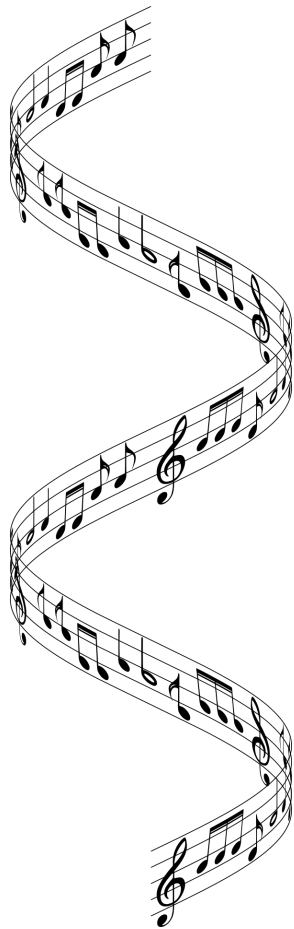
Deep learning is a collection of statistical techniques of machine learning for learning feature hierarchies that are actually based on artificial neural networks.



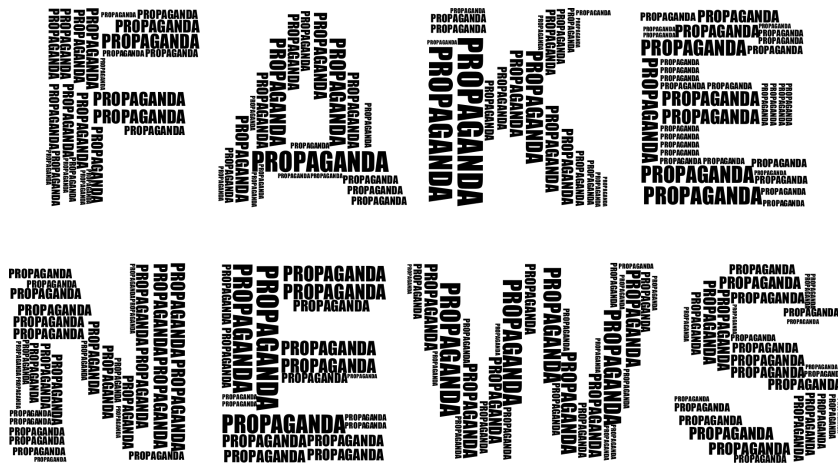
Why Deep Learning?



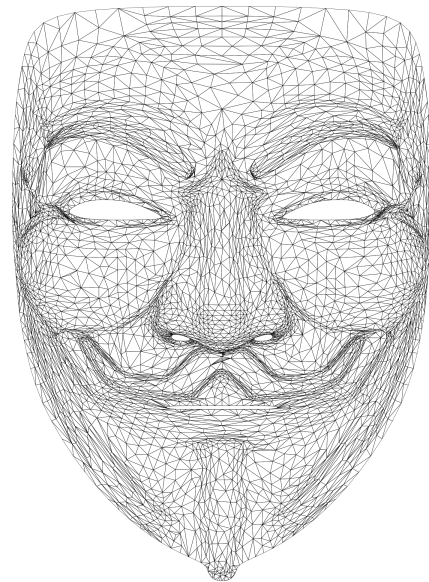
Images



Audio - Speech & Music



Language - Text



3D - Graphs, Point-clouds,
Mesh

Why Deep Learning ?

Higher Order Problem

- Require complex decision boundaries
- Require special constructs to process these modalities

Classical machine learning

- SVM, NB, LR, Ensemble learning : Not Enough

We live in data-age !

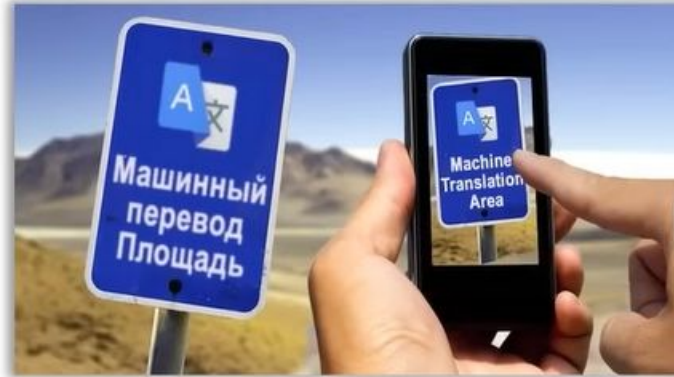
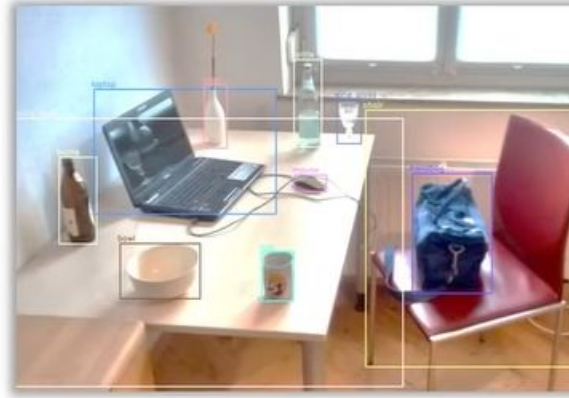
- As we feed more data, classical ML model don't improve.

Deep Learning was first theorized in the 1980s.

- It requires large amounts of labeled data
- It requires significant computational power (high performing GPUs)



Deep Learning Applications

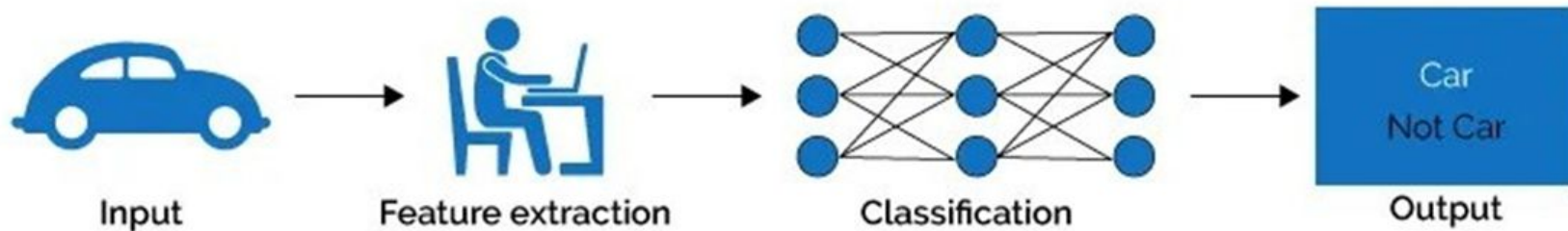


Deep Learning Vs. Machine Learning

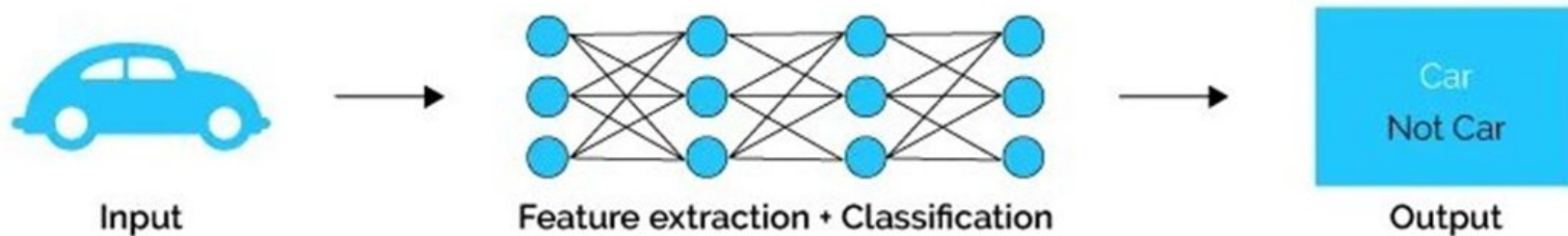
	Deep Learning	Machine Learning
Data	Needs a big dataset	Performs well with a small to a medium dataset
Hardware requirements	Requires machines with GPU	Works with low-end machines
Engineering peculiarities	Needs to understand the basic functionality of the data	Understands the features and how they represent the data
Training time	Long	Short
Processing time	A few hours or weeks	A few seconds or hours
Number of algorithms	Few	Many

Deep Learning Vs. Machine Learning

Machine Learning



Deep Learning



It's time for case study

SALMON

SHRIMP

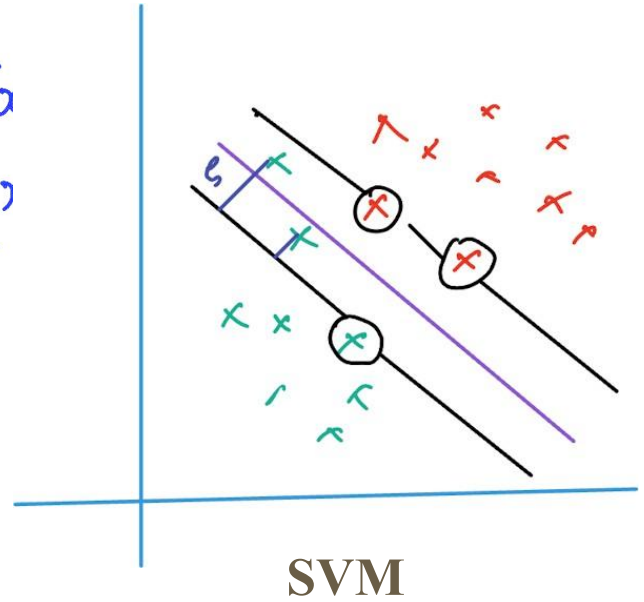
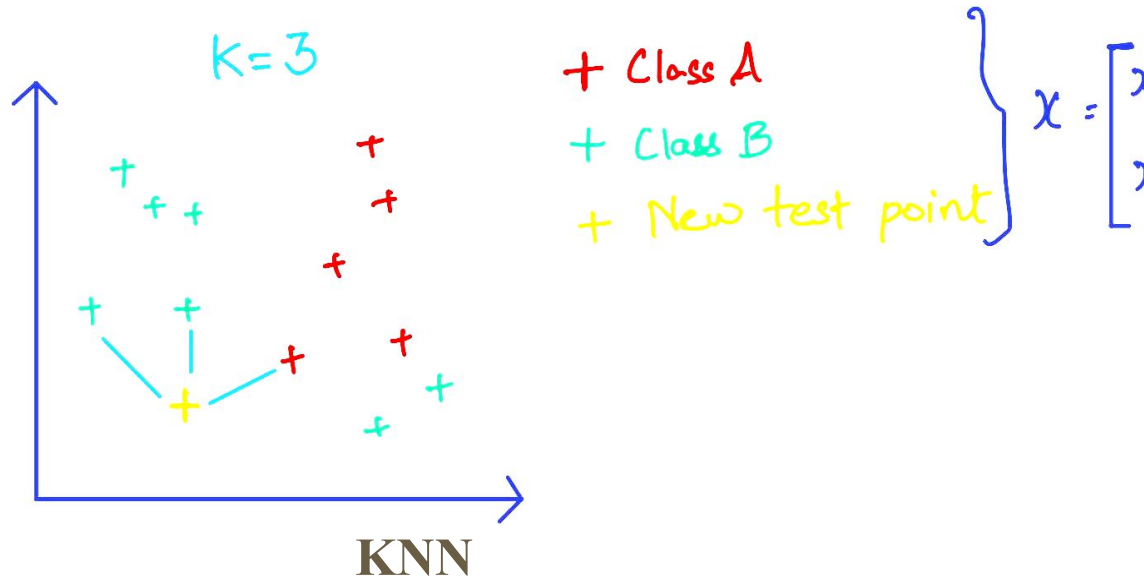


Let's build a 2-class classifier

Width	Height	Lightness	Fish_Type
2	3	1.1	Salmon (1)
1	5	0.8	Salmon (1)
0.5	0.6	0.6	Shrimp (-1)
0.9	0.5	0.7	Shrimp (-1)

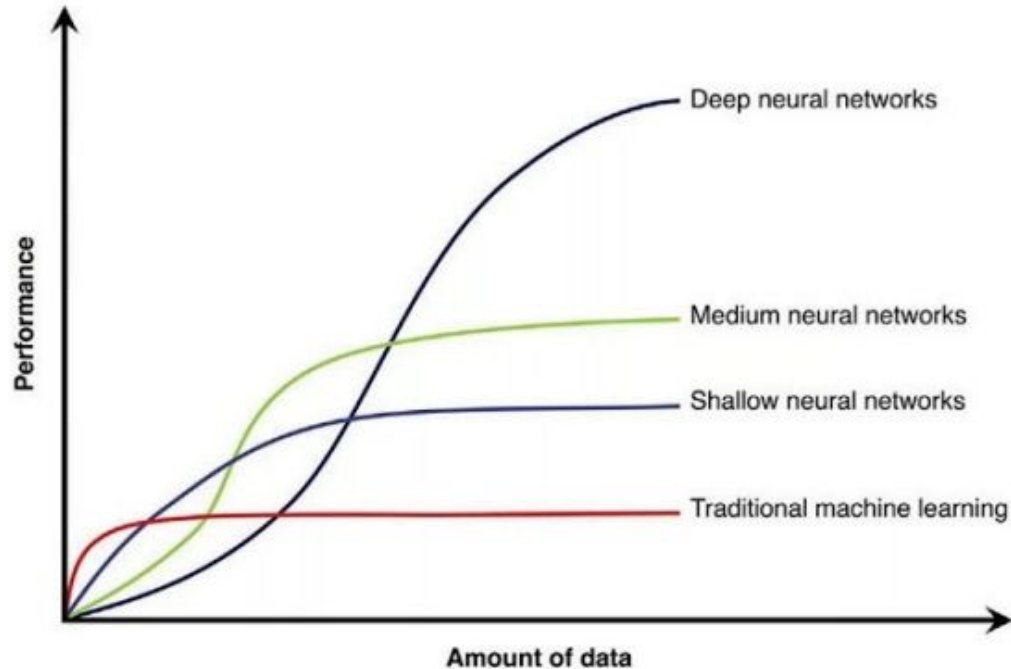


Classical Methods



Classical Methods will work for small data.
What if such 200k rows are there?

Why Deep Learning ?

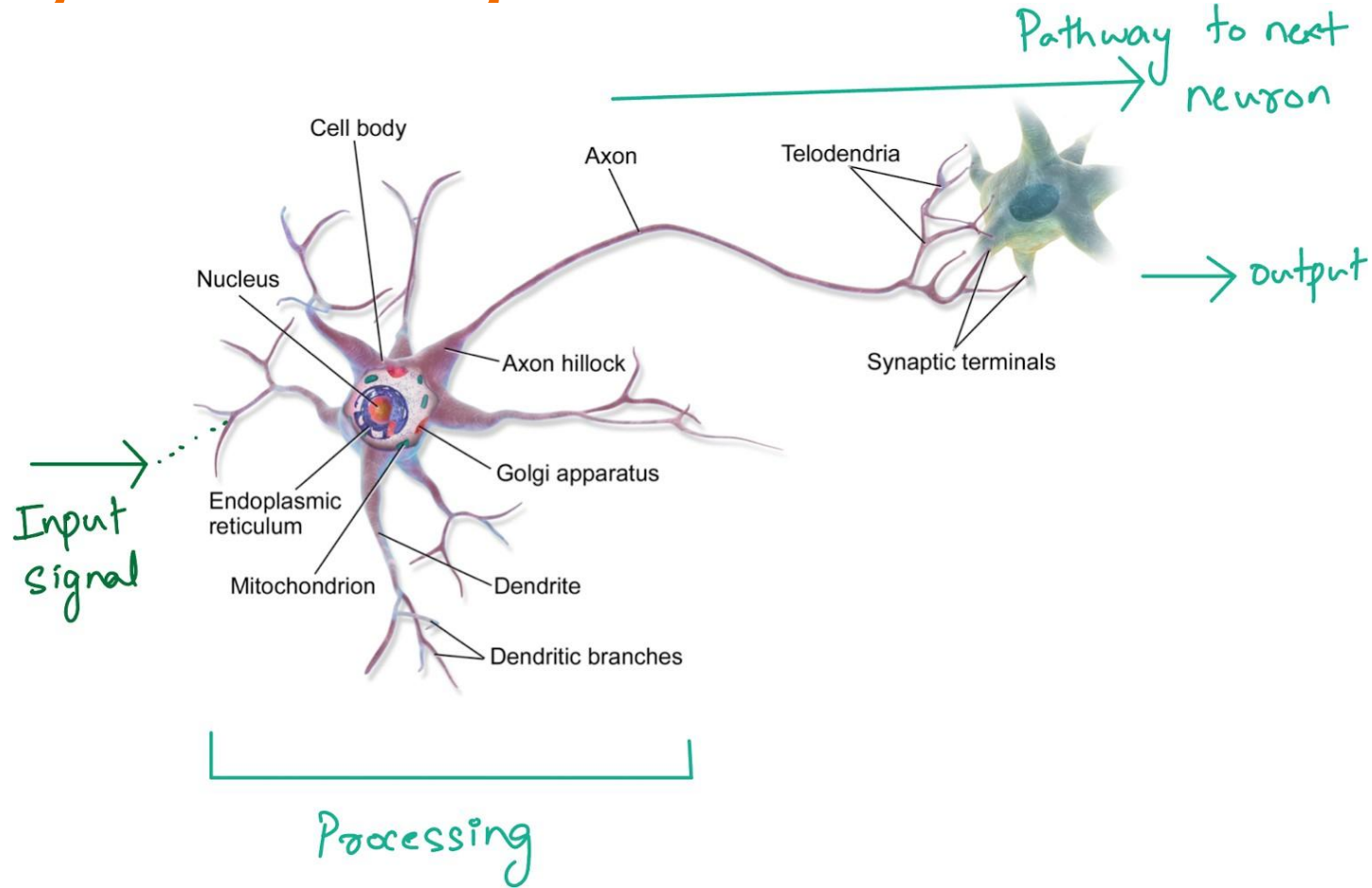


Why Now?

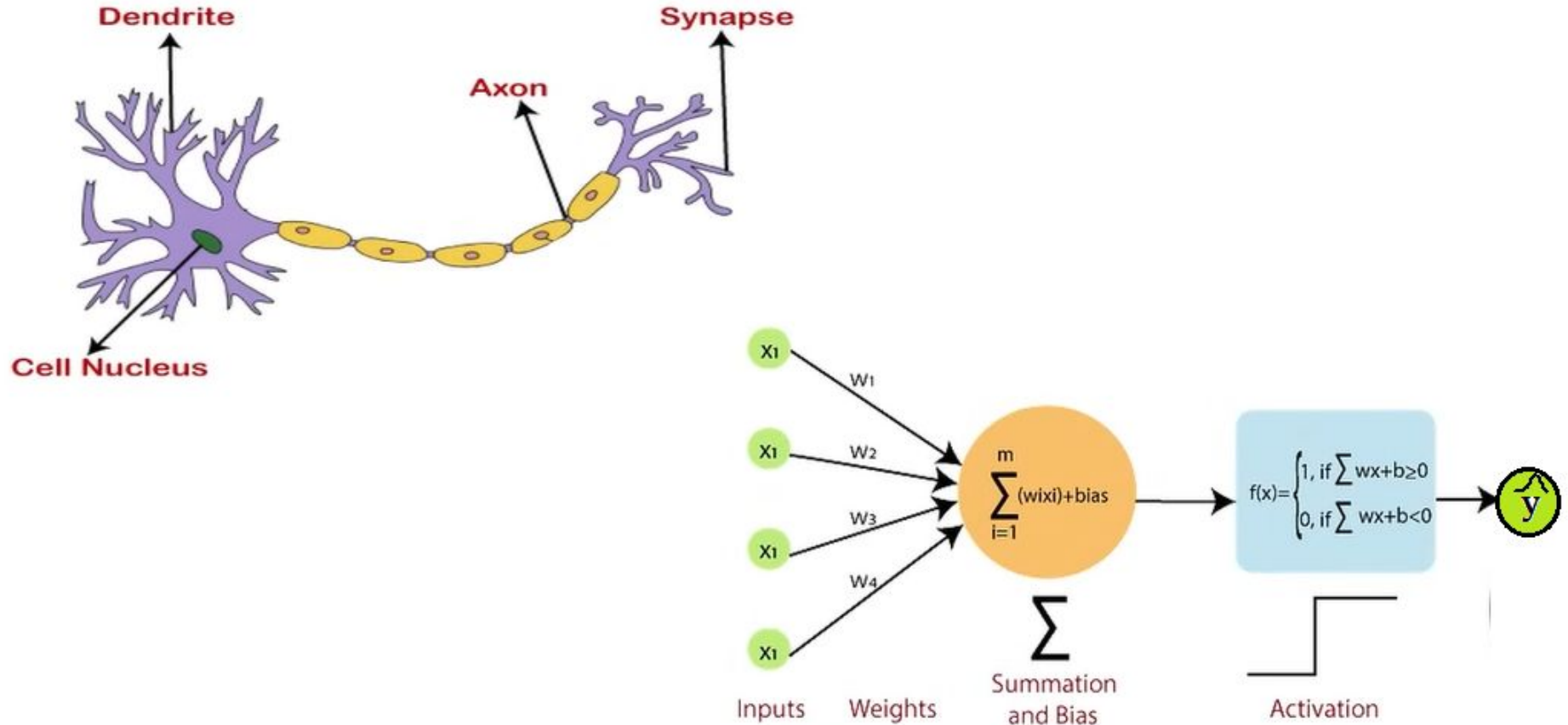
- *Algorithm Advancements*
- *GPU Computing*
- *Availability of Larger Training Data*

Deep learning is computationally expensive compared to machine learning, which also means that it requires a lot of time to process.

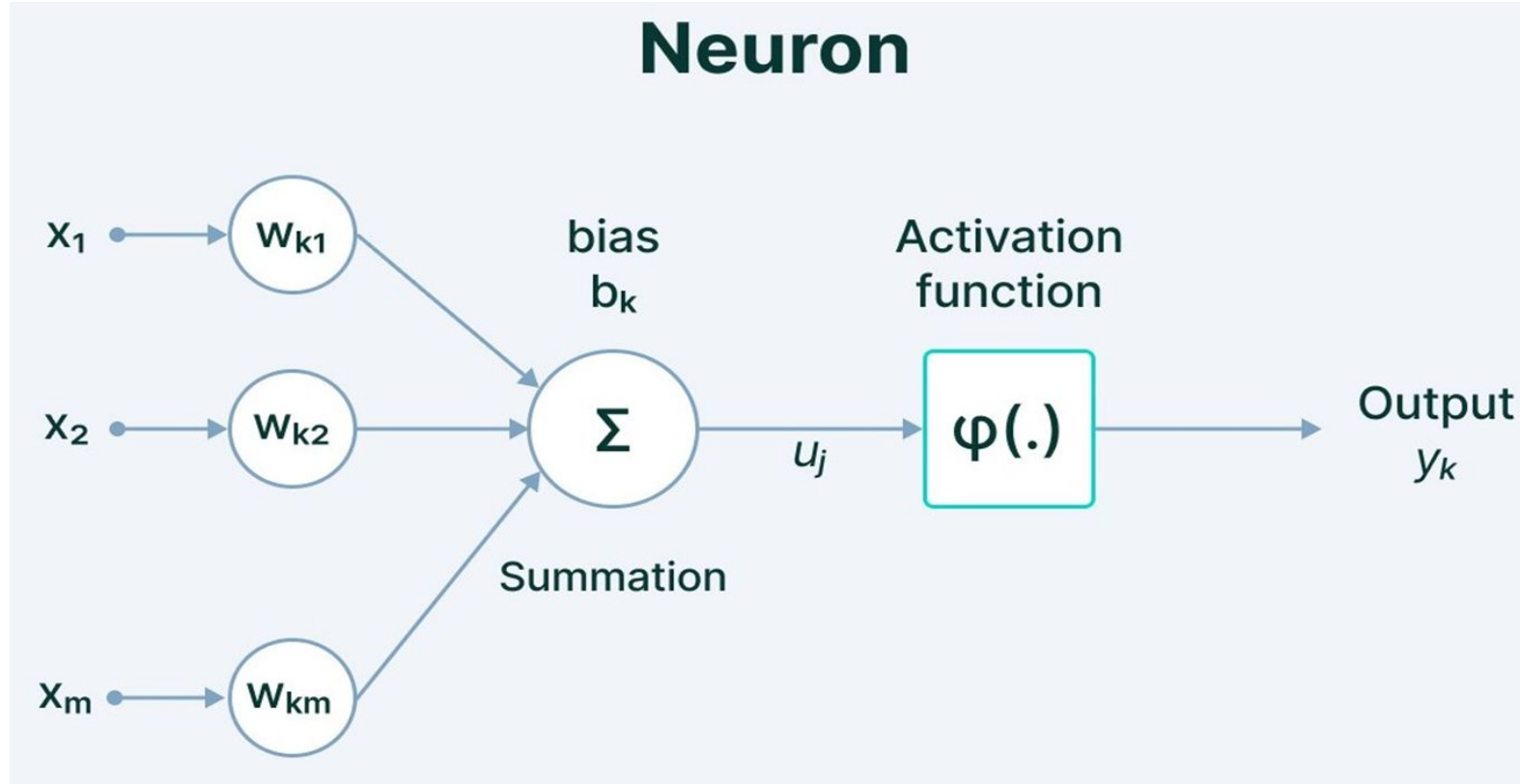
Story Starts : Biological Neurons



Can we use this idea to build a neural classifier?

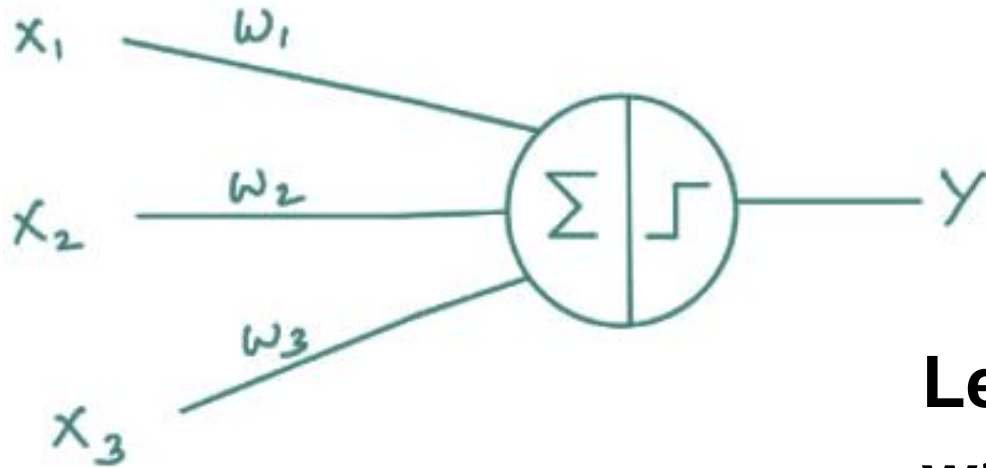


Then came - Artificial Neuron



<https://youtu.be/rEDzUT3ymw4?feature=shared>

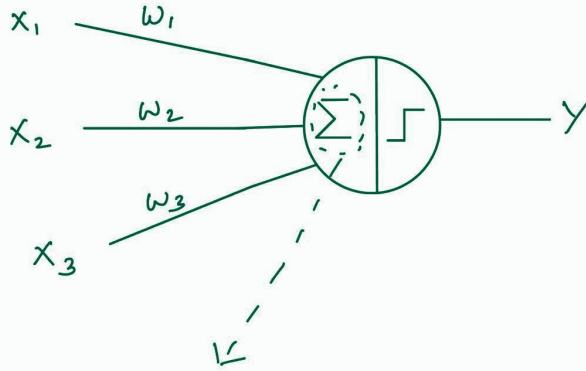
Artificial Neuron



Learnable Parameters:
 w_1, w_2, w_3, b

$$\text{Step 1) } w_1 x_1 + w_2 x_2 + w_3 x_3 + b = w^T x + b$$

Artificial Neuron



Step 1) $w_1 x_1 + w_2 x_2 + w_3 x_3 + b = w^T x + b$

Step 2) $\text{sgn}(w^T x + b)$

$$\text{sgn} = \begin{cases} -1 & \text{if } w^T x + b < 0 \\ 0 & \text{if } w^T x + b = 0 \\ 1 & \text{if } w^T x + b > 0 \end{cases}$$

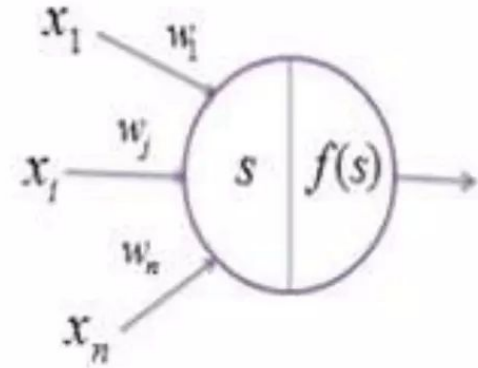
(step or sign function)



Fish_type	Width	Height	Lightness
Salmon (1)	2	3	1.1
Salmon (1)	1	5	0.8
Shrimp (-1)	0.5	0.6	0.6
Shrimp (-1)	0.9	0.5	0.7

What is Perceptron?

- A perceptron **models** a neuron
- It receives **n inputs** (feature vector)
- It sum those inputs , calculated, then output
- Used for **linear** or **binary classification**



Summation

$$s = \sum_{i=1}^n w_i \cdot x_i$$

Transformation



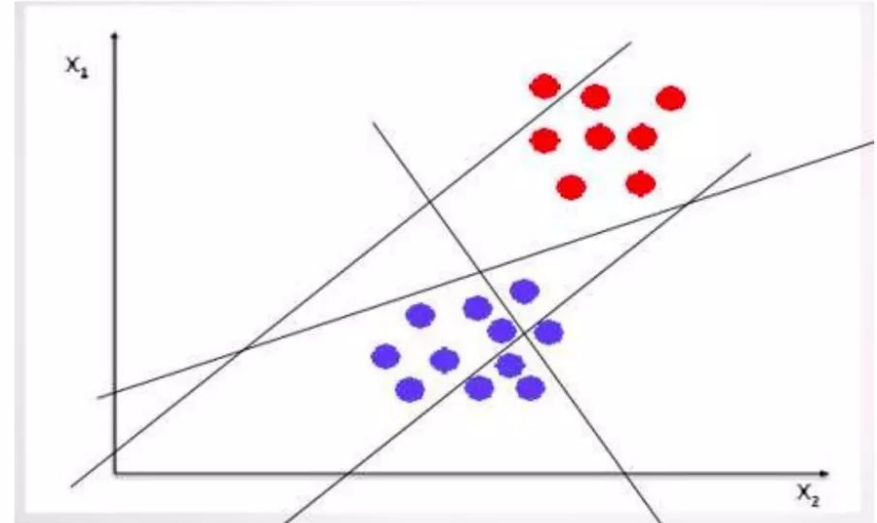
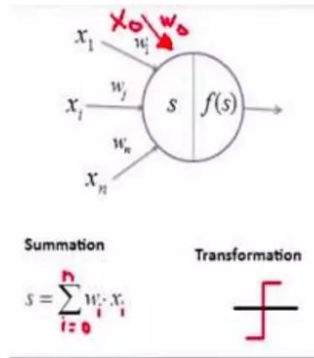
Perceptron

- The perceptron consists of weights, the summation processor, and an activation function
- A perceptron takes a weighted sum of inputs and outputs:

	Output
$w_1x_1 + w_2x_2 + \dots + w_nx_n > \theta$	1
$w_1x_1 + w_2x_2 + \dots + w_nx_n \leq \theta$	0

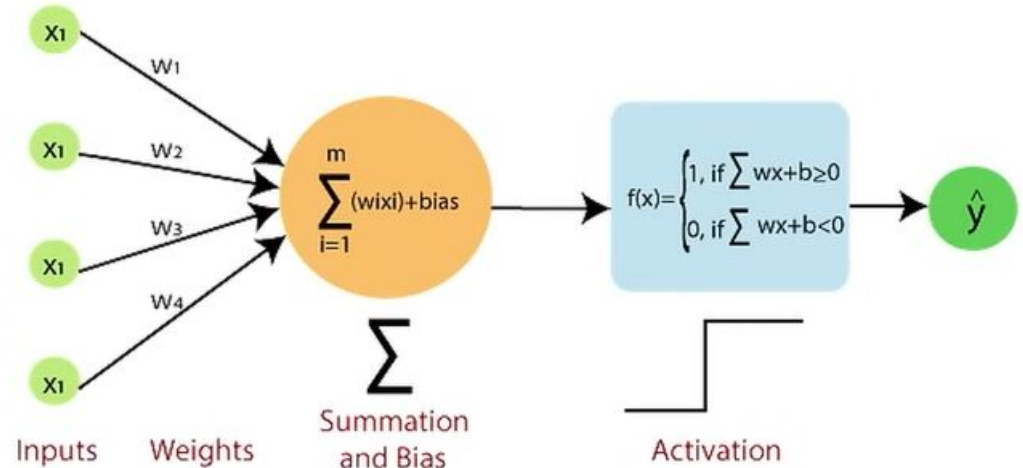
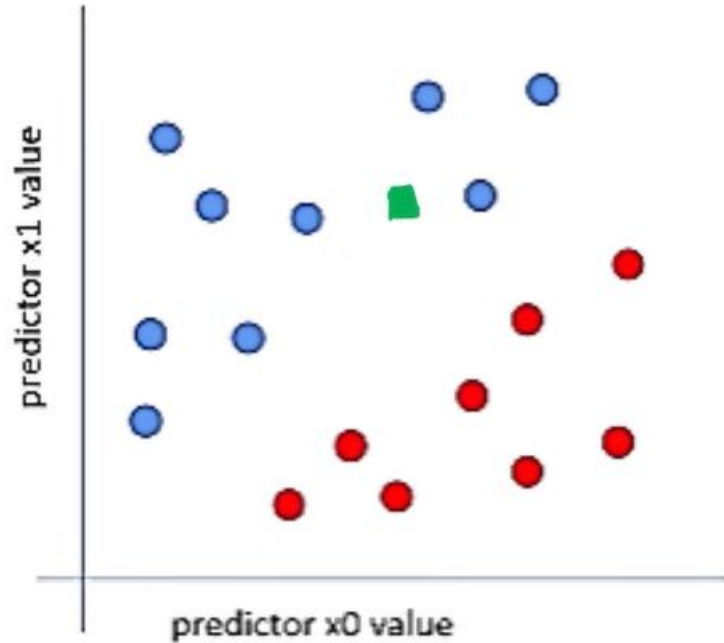
Weight & Bias

- Bias can also be treated as another input
 - The bias allow to **shift the line**
- The weights **determine the slope**



Single Layer Perceptron

Single Layer Perceptron



$$\hat{y} = x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 + b$$

$$\hat{y} = W_T \cdot X + b$$

How it works?

- Deep Neural Networks have multiple layers of interconnected artificial neurons or nodes that are stacked together.
- Each of these nodes has a simple mathematical function - usually a linear function that performs extraction and mapping of information.
- There are three layers to a deep neural network: the input layer, hidden layers, and the output layer.

Can we use this idea to build a neural classifier?

A lot like a function

$$y = f(x)$$

↑ ↑
o/p i/p

→ processing

X: Features of a Fish

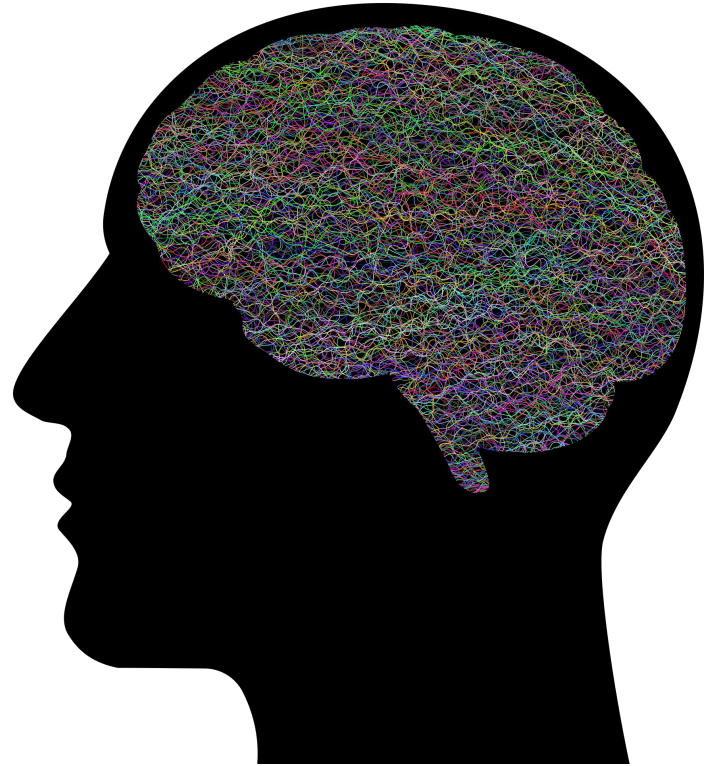
Y: Type of Fish

Tinker With a Neural Network

A Neural Network Playground - TensorFlow

But, our brains contain not just 1 neuron!

Can we do better?



Multi Layer Perceptron

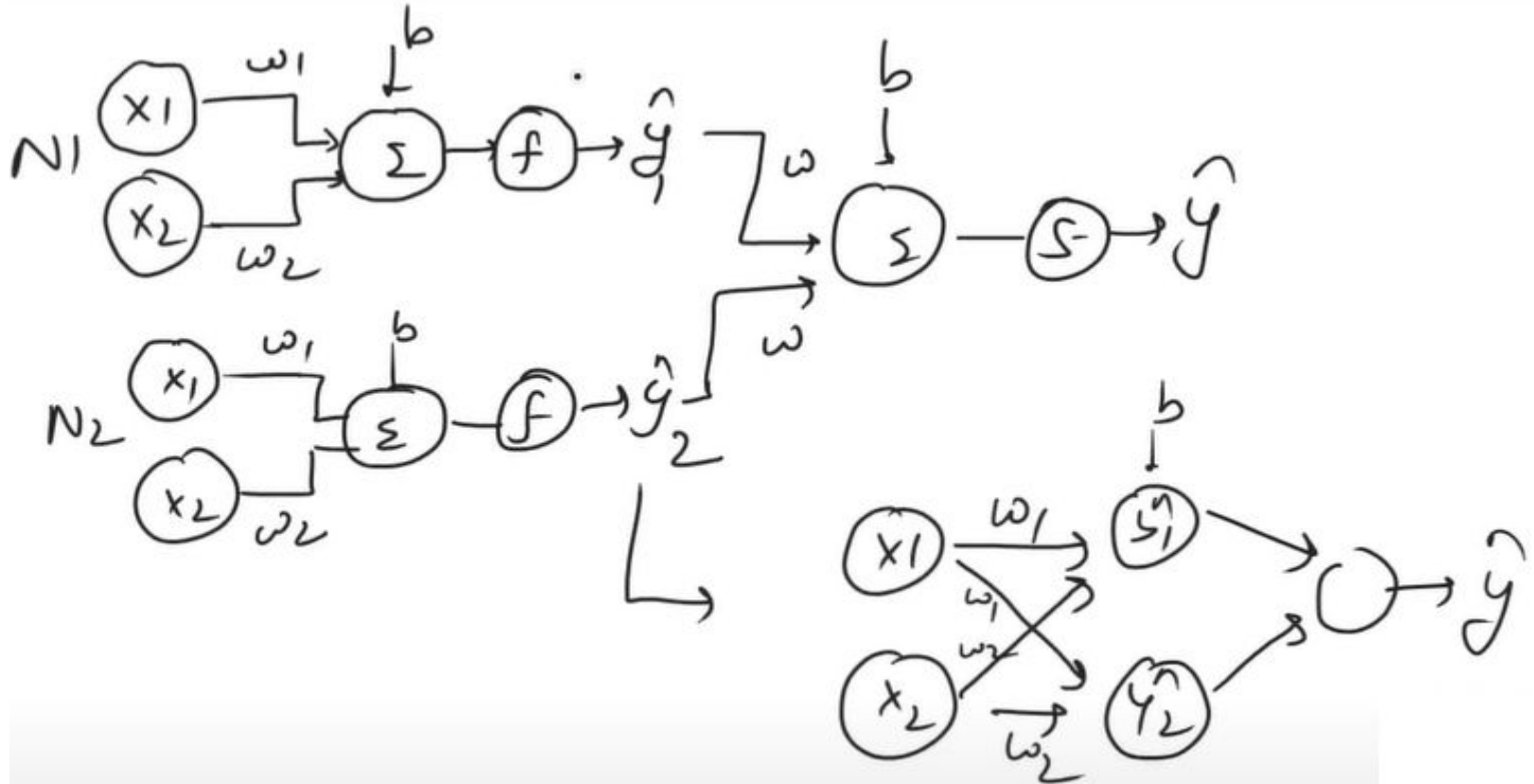
Multi Layer Perceptron

- A single neuron hardly makes a dent in the face of a complex calculation. What is needed is a hierarchical mesh of neurons.
- Multilayer perceptron (MLP) is a complex structure that helps a machine learn a much more sophisticated decision boundary. It has applications in stock price prediction, image classification, spam detection, sentiment analysis, data compression, etc.

A multilayer perceptron has three segments:

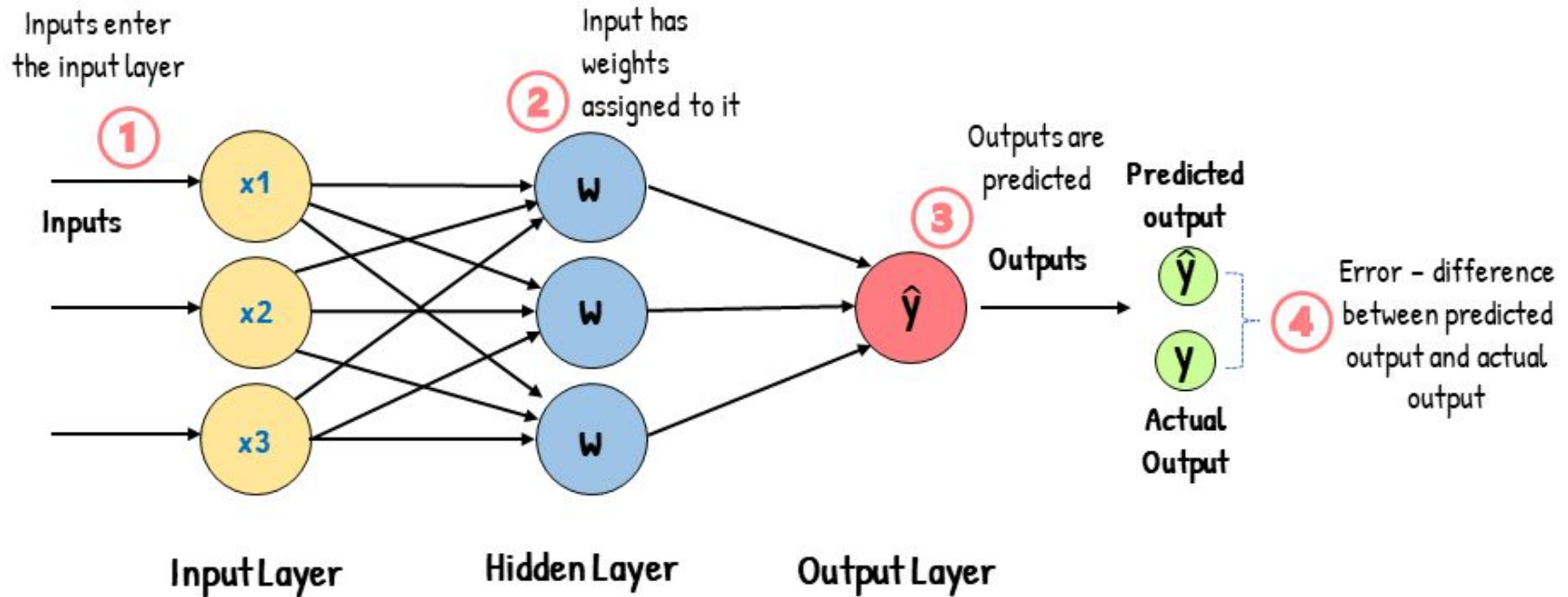
- Input layer, where data is fed into the network.
- Hidden layer, where all the mathematical calculations are done using weights, biases, and the data.
- Output layer, where meaningful results are obtained from.

Multi Layer Perceptron



Forward Propagation

Feed-Forward Neural Network



Forward Propagation



Back Propagation



Backward Propagation

Backward Propagation

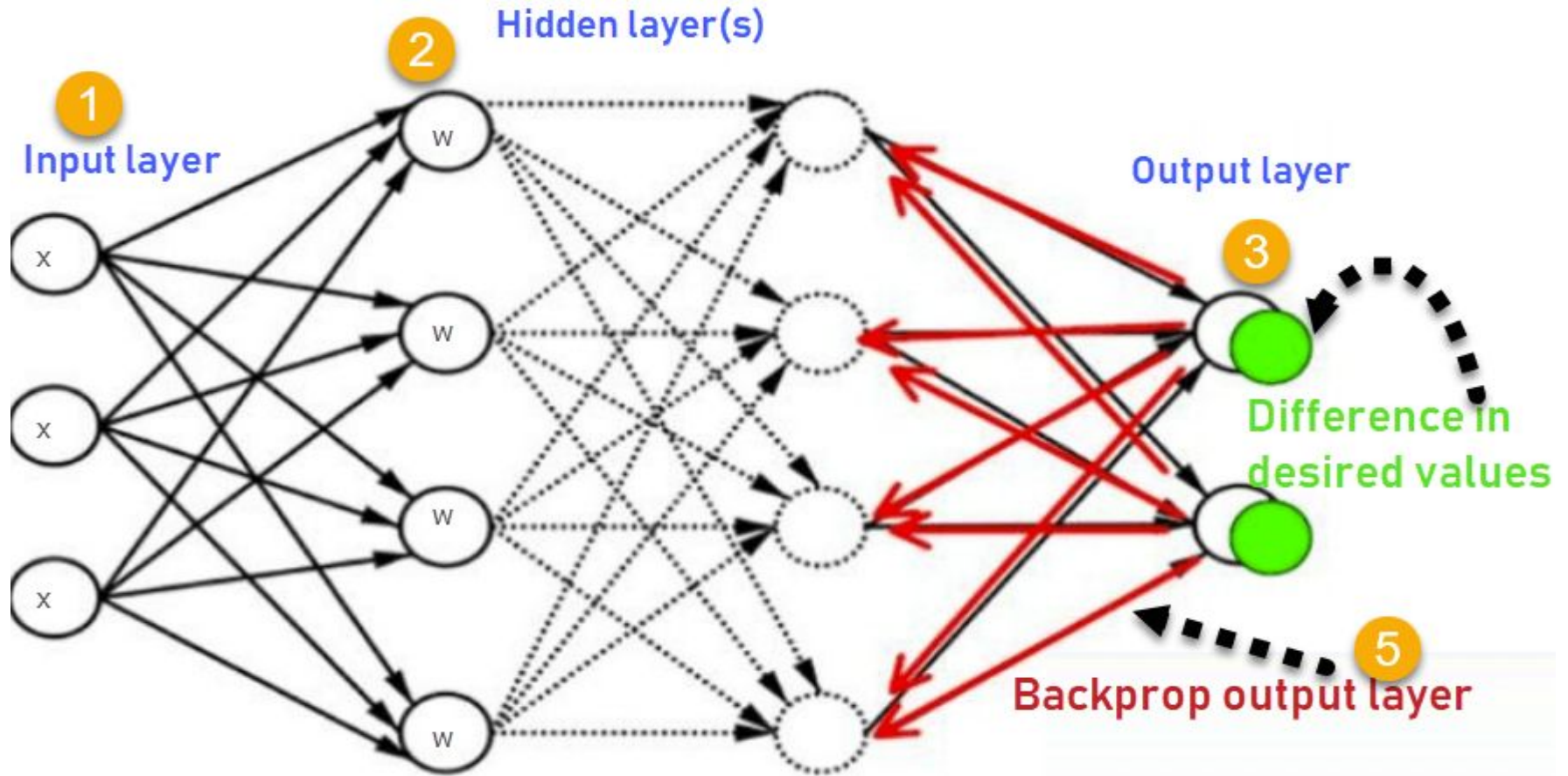
1. Feedforward | Mean Squared Error (MSE) computed



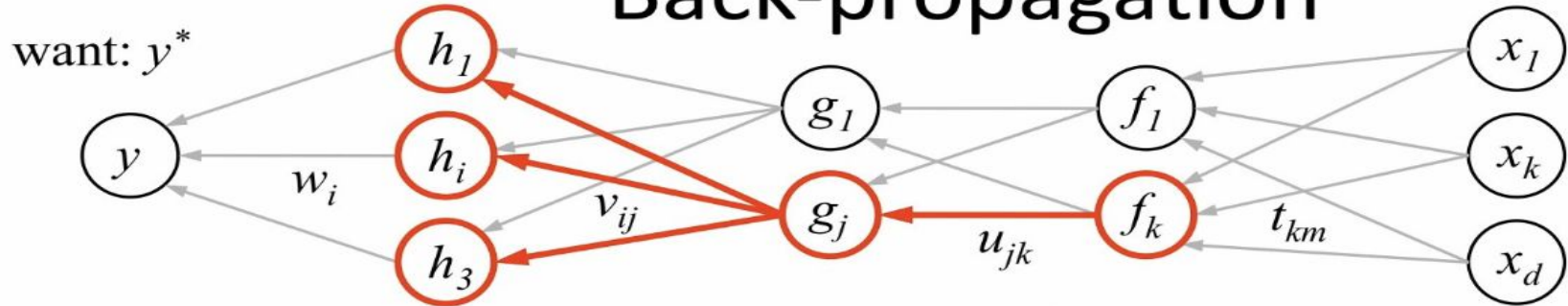
The diagram consists of two horizontal arrows. The top arrow points to the right and is labeled '1. Feedforward | Mean Squared Error (MSE) computed'. The bottom arrow points to the left and is labeled '2. Backpropagation | Gradient is computed'. The arrows are gray and have a slight 3D effect with a shadow.

2. Backpropagation | Gradient is computed

Backward Propagation



Back-propagation



1. receive new observation $\mathbf{x} = [x_1 \dots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer $1 \dots L$
compute g_j based on units f_k from previous layer: $g_j = \sigma \left(u_{j0} + \sum_k u_{jk} f_k \right)$
3. get prediction y and error $(y - y^*)$
4. **back-propagate error:** for each unit g_j in each layer $L \dots 1$

(a) compute error on g_j

$$\underbrace{\frac{\partial E}{\partial g_j}}_{\text{should } g_j \text{ be higher or lower?}} = \sum_i \underbrace{\sigma'(h_i)}_{\text{how } h_i \text{ will change as } g_j \text{ changes}} \underbrace{v_{ij}}_{\text{was } h_i \text{ too high or too low?}} \underbrace{\frac{\partial E}{\partial h_i}}_{\text{was } h_i \text{ too high or too low?}}$$

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \underbrace{\frac{\partial E}{\partial g_j}}_{\text{do we want } g_j \text{ to be higher/lower}} \underbrace{\sigma'(g_j) f_k}_{\text{how } g_j \text{ will change if } u_{jk} \text{ is higher/lower}}$$

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

Increasing the number of hidden layers

Increasing the number of hidden layers leads to two known issues:

Vanishing gradients:

- As we add more and more hidden layers, backpropagation becomes less and less useful in passing information to the lower layers.
- In effect, as information is passed back, the gradients begin to vanish and become small relative to the weights of the networks.

Overfitting:

- Overfitting describes the phenomenon of fitting the training data too closely, maybe with hypotheses that are *too* complex.
- In such a case, your learner ends up fitting the training data really well, but will perform much, much more poorly on real examples.

A Quick Summary

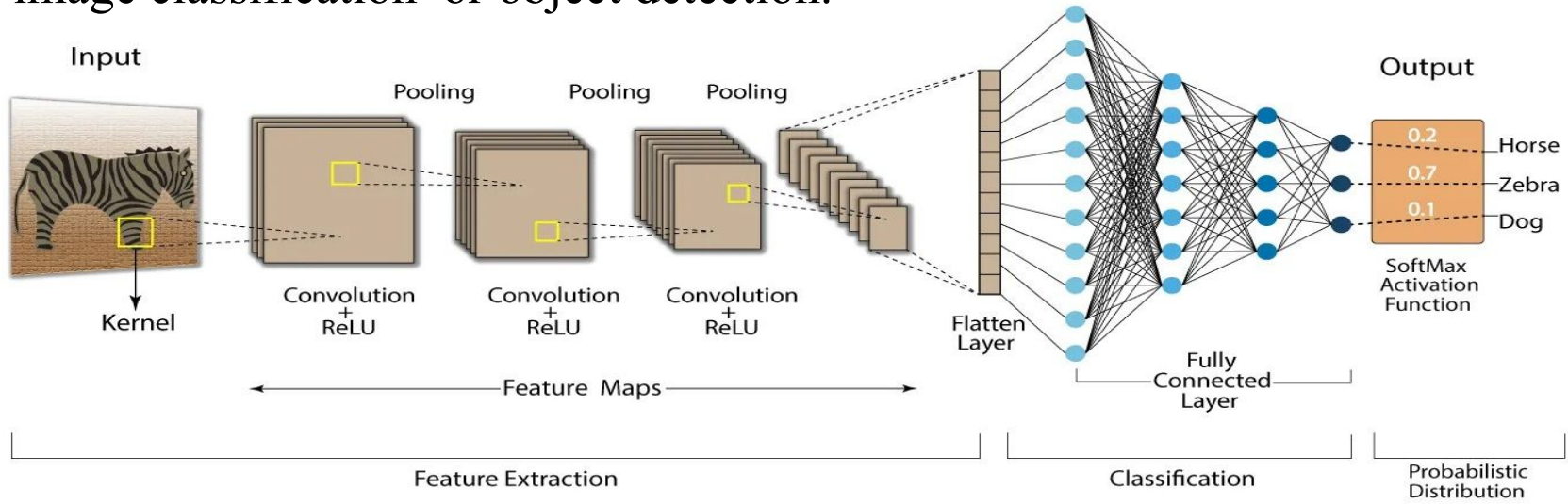
- ML requires data preprocessing, which involves human intervention.
- The neural networks in DL are capable of extracting features; hence no human intervention is required.
- DL can process unstructured data.
- DL is based on representative learning i.e., finding and extracting vital information or patterns that represent the entire dataset.
- DL is computationally expensive and time-consuming.

Types of Neural Network

- Artificial Neural Network
- Convolutional Neural Network
- Recurrent Neural Network
- Generative Adversarial Network

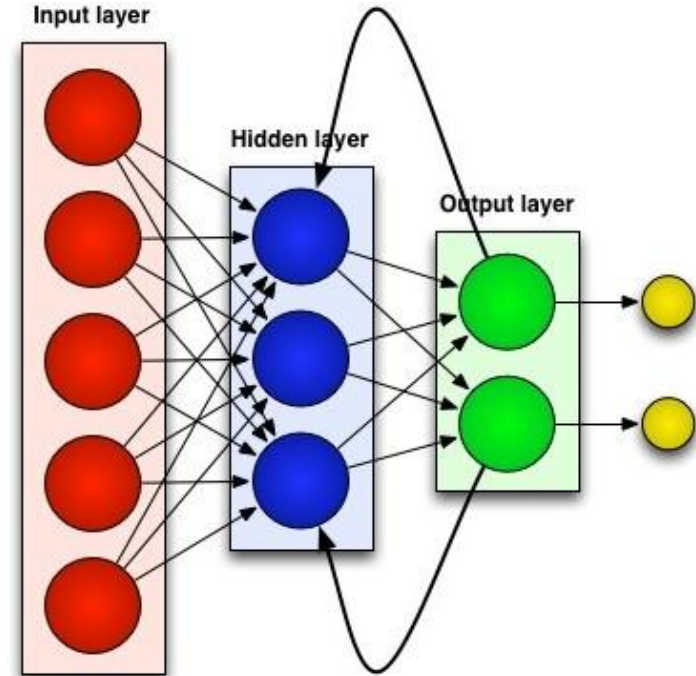
CNN

- The Convolutional Neural Networks or CNNs are primarily used for tasks related to computer vision or image processing.
- CNNs are extremely good in modeling spatial data such as 2D or 3D images and videos.
- They can extract features and patterns within an image, enabling tasks such as image classification or object detection.



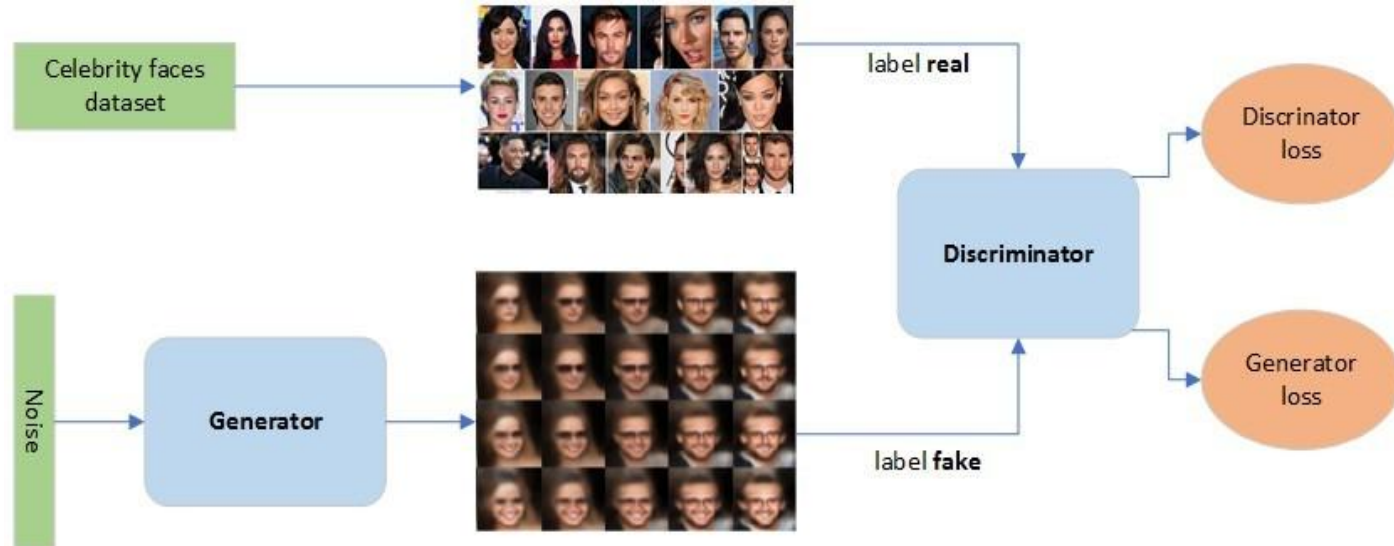
RNN

- The Recurrent Neural Networks or RNN are primarily used to model sequential data, such as text, audio, or any type of data that represents sequence or time.
- They are often used in tasks related to natural language processing (NLP).



GAN

- Generative adversarial networks or GANs are frameworks that are used for the tasks related to unsupervised learning.
- This type of network essentially learns the structure of the data, and patterns in a way that it can be used to generate new examples, similar to that of the original dataset.



Transformers

- Transformers are the new class deep learning model that is used mostly for the tasks related to modeling sequential data, like that in NLP.
- It is much more powerful than RNNs and they are replacing them in every task.
- Recently, transformers are also being applied in computer vision tasks and they are proving to be quite effective than the traditional CNNs.

Deep Learning: Limitations

- Data availability
- The complexity of the model Lacks global generalization
- Incapable of Multitasking Hardware dependence

Deep Learning: Limitations

Data availability

- Deep learning models require a lot of data to learn the representation, structure, distribution, and pattern of the data.
- If there isn't enough varied data available, then the model will not learn well and will lack generalization (it won't perform well on unseen data).
- The model can only generalize well if it is trained on large amounts of data.

Deep Learning: Limitations

The complexity of the model

- Designing a deep learning model is often a trial and error process.
- A simple model is most likely to underfit, i.e. not able to extract information from the training set, and a very complex model is most likely to overfit, i.e., not able to generalize well on the test dataset.
- Deep learning models will perform well when their complexity is appropriate to the complexity of the data.

Deep Learning: Limitations

Lacks global generalization

- A simple neural network can have thousands to tens of thousands of parameters.
- The idea of global generalization is that all the parameters in the model should cohesively update themselves to reduce the generalization error or test error as much as possible. However, because of the complexity of the model, it is very difficult to achieve zero generalization error on the test set.
- Hence, the deep learning model will always lack global generalization which can at times yield wrong results.

Deep Learning: Limitations

Incapable of Multitasking

- Deep neural networks are incapable of multitasking.
- These models can only perform targeted tasks, i.e., process data on which they are trained. For instance, a model trained on classifying cats and dogs will not classify men and women.
- Furthermore, applications that require reasoning or general intelligence are completely beyond what the current generation's deep learning techniques can do, even with large sets of data.

Deep Learning: Limitations

Hardware dependence

- As mentioned before, deep learning models are computationally expensive.
- These models are so complex that a normal CPU will not be able to withstand the computational complexity.
- However, multicore high-performing graphics processing units (GPUs) and tensor processing units (TPUs) are required to effectively train these models in a shorter time.
- Although these processors save time, they are expensive and use large amounts of energy.



Thank You

